

A man with a beard, wearing a light blue shirt with 'KRA' on the chest, is sitting at a desk. He is using a pen to draw on a set of architectural blueprints. A wooden ruler is placed horizontally across the bottom left of the blueprints. The background is dark and out of focus.

Task Decomposition

การแตกงานซับซ้อนออกเป็นขั้นตอนย่อยให้ระบบ agent ทำได้

Domain 1 · Task 1.6 — Claude Certified Architect

Task Decomposition คืออะไร?

- คือการ **แบ่งงานซับซ้อน** ออกเป็นชิ้นงานย่อยที่จัดการได้สำหรับระบบ agentic

● Pattern 1

Fixed Sequential Pipelines

● Pattern 2

Dynamic Adaptive Decomposition

⚠ **Failure mode เฉพาะตัว: attention dilution**

สามอย่างนี้คือ **แกนของทั้ง Task**

Pattern 1: Fixed Sequential Pipelines

Prompt Chaining

นิยาม

แบ่งงานเป็นขั้นตอนที่กำหนดไว้ล่วงหน้า (**predetermined steps**) แล้วรันตามลำดับตายตัว

output ของ Step 1 → ป้อนเข้า Step 2 → ป้อนเข้า Step 3 โดยไม่เบี่ยงเบนตามสิ่งที่ค้นพบระหว่างทาง

ตัวอย่าง: Code Review Pipeline

Cross file

1. วิเคราะห์แต่ละไฟล์แยกกัน (style, bugs, complexity)

2. รวมตรวจสอบข้ามไฟล์ (data flow, API consistency, imports)

3. รวบรวมเป็นรายงานเดียว

Fixed Pipeline: จุดแข็ง vs จุดอ่อน

🎯 เหมาะกับ

งานที่**คาดเดาได้** มีโครงสร้างชัดเจน รู้ขั้นตอนล่วงหน้า

code review

document processing

data extraction

compliance checks

✓ จุดแข็ง

ความสม่ำเสมอ (consistency) · น่าเชื่อถือ · debug
ง่าย · monitor ได้ผ่าน log ที่ละขั้น

✗ จุดอ่อน

ปรับตัวตามสิ่งที่ค้นพบระหว่างทาง**ไม่ได้**

intermediate discovery ไม่สามารถเปลี่ยนรูปขั้นตอนถัดไปได้

Pattern 2: Dynamic Adaptive Decomposition

นิยาม

สร้าง subtask ขึ้น**ตามสิ่งที่ค้นพบ**ในแต่ละช่วง (discoveries at each stage) — แผนวิวัฒนาการไปเรื่อย ๆ เมื่อ agent เข้าใจปัญหามากขึ้น

เป้าหมายระดับสูง → สำรวจเบื้องต้น → วางแผนตามสิ่งที่เจอ → ปรับขั้นตอนที่เหลือเมื่อมีข้อมูลใหม่

ตัวอย่าง: Legacy Codebase Testing

map โครงสร้าง → หาจุด high-impact → วางแผนเทสต์แบบจัดลำดับ → เริ่มเขียนเทสต์แล้ว**พบว่า Module A ขึ้นกับ Module B ที่ยังไม่มีเทสต์** → จัดลำดับใหม่ เทสต์ B ก่อน → ปรับต่อไปเรื่อย ๆ ตาม dependency ที่เจอ

Dynamic Decomposition: จุดแข็ง vs จุดอ่อน

🎯 เหมาะกับ

งาน**ปลายเปิด** (open-ended) ที่ยังไม่รู้ขอบเขต

legacy exploration

security audits

research

debug ระบบไม่คุ้นเคย

✓ จุดแข็ง

ปรับตามความซับซ้อนได้ · ค้นเจอปัญหาที่ไม่คาดคิด · ให้
ผลครบถ้วนแม้ขอบเขตยังไม่นิยาม

✗ จุดอ่อน

คาดเดาการทำงานยาก · เวลาเสร็จไม่แน่นอน · ประเมิน resource ยาก ·
debug ยากกว่า

หลักเลือก Pattern



Key Concept

เลือกตาม "ความรู้ล่วงหน้าเรื่องขั้นตอน" ไม่ใช่ตามความยากของงาน

รู้ขั้นตอนแน่นอน + input มีโครงสร้าง

→ **Fixed pipeline** (เอาความสม่ำเสมอ)

multi-file code review → **Fixed** (flow คาดเดาได้)

document extraction → **Fixed** (field/format กำหนดแล้ว)

ปลายเปิด ขอบเขตไม่นิยาม

→ **Dynamic decomposition** (เอาความยืดหยุ่น)

legacy exploration → **Dynamic** (ปัญหาโผล่ระหว่างสำรวจ)

debug ระบบไม่คุ้น → **Dynamic** (ยังไม่รู้ root cause)

Attention Dilution Problem

★ ออกสอบบ่อย

นิยาม

failure mode ที่เกิดเมื่อประมวลผล **item หลายชิ้นพร้อมกันในรอบเดียว** ทำให้ความลึกของการวิเคราะห์ไม่สม่ำเสมอ — บาง item ตรวจละเอียด บาง item พลาดปัญหาไป

อาการ

- ไฟล์แรก ๆ ได้ feedback ละเอียด ไฟล์หลัง ๆ ตื้นลงเรื่อย ๆ
- pattern เดียวกันถูกฟ้องว่าผิดในไฟล์หนึ่ง แต่ผ่านในอีกไฟล์
- bug ชัด ๆ ในไฟล์ท้าย ๆ หลุด ขณะที่เรื่องเล็กในไฟล์ต้น ๆ กลับถูกจับ

สาเหตุ

โมเดลจัดสรร **attention ที่มีจำกัด** ให้ทุก item ในบริบท — ยิ่ง item เยอะ attention ต่อชิ้นยิ่งน้อย และ item ต้น ๆ ได้ focus มากเกินสัดส่วน

วิธีแก้: Multi-Pass Architecture

 Key Concept

attention dilution เป็นปัญหาเชิงสถาปัตยกรรม (architectural) ไม่ใช่ปัญหาความสามารถของโมเดล

แก้ด้วย**โครงสร้าง** ไม่ใช่ด้วยฮาร์ดแวร์

Layer 1 — Per-item local pass

วิเคราะห์แต่ละไฟล์/เอกสารทีละ**ชิ้น** ให้แต่ละชิ้นได้ **attention budget** เต็ม

Layer 2 — Cross-item integration pass

การวิเคราะห์แยกอีกรอบที่มอง**ความสัมพันธ์ระหว่างชิ้น** — data flow, ความสอดคล้องของ pattern ข้าม item ทั้งหมด



Case Study: รีวิวโค้ด 14 ไฟล์

อาการ (แบบ single-pass)

ไฟล์ 1-5 ได้ feedback ละเอียดมีเลขบรรทัด · ไฟล์ 6-9 ปานกลาง · ไฟล์ 10-14 **ตื้นมาก**พลาดทั้ง null pointer bug และ SQL injection
forEach loop ถูกฟ้องว่าไม่มีประสิทธิภาพในไฟล์ 3 แต่โค้ดเดียวกัน**ถูกอนุมัติในไฟล์ 11**

สาเหตุ

attention dilution จากการยัดหลายไฟล์เข้ารอบเดียว

✅ วิธีแก้

ทำ **per-file pass 14 รอบแยกกัน** + **cross-file integration pass 1 รอบ** → ได้ความลึกสม่ำเสมอทุกไฟล์ และจับความไม่สอดคล้องของ pattern ข้ามไฟล์ได้



Exam Traps ที่ต้องระวัง

1. เสนอ অপเกรดโมเดล / ขยาย context window

× attention dilution เป็นปัญหาสถาปัตยกรรม — ฮาร์ดแวร์ที่ดีขึ้นไม่แก้ข้อจำกัดเชิงโครงสร้างของ single-pass

2. เชื่อว่า prompt ที่ดีขึ้นแทน multi-pass ได้

× prompt ดีขึ้นเพิ่มคุณภาพเฉลี่ย แต่การันตีการจัดสรร attention สม่ำเสมอทุก item ไม่ได้

3. เอา fixed pipeline ไปใช้กับงานปลายเปิด

× ลำดับตายตัวตอบสนองสิ่งที่ค้นพบระหว่างสำรวจไม่ได้

4. Batching โดยไม่มี cross-file integration

× ลด dilution ภายใน batch ได้ แต่พลาด data flow และ pattern ที่ข้าม batch



Practice Scenario

ลองคิดก่อนดูเฉลย

โจทย์: agent รีวไรต์โค้ดให้ feedback ละเอียดกับ 5 ไฟล์แรก แต่พลาด bug ชัด ๆ ในไฟล์ 10-14 และประเมิน pattern `forEach` เดียวกันขัดแย้งกันเองข้ามไฟล์ — **ควรแก้อย่างไร?**

- A. ลดเหลือ batch ละ 5 ไฟล์ แล้วรันเรียงกันไป
- B. อัปเดตขนาด context window ของโมเดล
- C. เสริม system prompt ให้เน้นย้ำว่าต้องละเอียดเท่ากันทุกไฟล์
- D. ทำ per-file local pass แยกทุกไฟล์ + cross-file integration pass อีกหนึ่งรอบ

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อดูเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option D

ทำ **per-file local pass** แยกทุกไฟล์เพื่อให้แต่ละไฟล์ได้ attention เต็ม แล้วตามด้วย **cross-file integration pass**
— เป็นการแก้ไขโครงสร้างที่ตรงกับต้นเหตุ attention dilution

B ผิด — เป็นปัญหาสถาปัตยกรรม ไม่ใช่ context window

C ผิด — prompt การันตีการจัดสรร attention เสมอไม่ได้

A ใกล้เคียงแต่ยังไม่พอ — batching ad dilution ภายใน batch ได้ แต่ถ้าไม่มี cross-file integration ก็ยังพลาด data flow และ pattern ที่ข้าม batch (เช่น `forEach` ที่ประเมินขัดกันคนละ batch)



Build Exercise

แบบฝึก 60 นาที — Multi-Pass Code Review Pipeline

1. agent รับ directory ที่มีไฟล์ 10+ ไฟล์

2. ทำ single-pass baseline แล้วบันทึกผล

3. ทำ per-file local pass ที่ให้ feedback มีโครงสร้าง (จำนวน bug, severity, เลขบรรทัด)

4. ทำ cross-file integration pass ตรวจ data flow, API consistency, pattern consistency

5. เปรียบผล single-pass vs multi-pass

6. บันทึก artifact ของ attention dilution (pattern เดียวกันถูกปฏิบัติต่างกัน) เพื่อยืนยันว่า multi-pass แก้ปัญหาเชิงโครงสร้างได้จริง

Domain 1 · Task 1.6 — Task Decomposition 



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

agent รีวิวโค้ดประมวลผล 14 ไฟล์ ให้ feedback ละเอียดกับ 5 ไฟล์แรก แต่พลาด bug ชัด ๆ ในไฟล์ 10-14 และยัง flag ว่า forEach loop ไม่มีประสิทธิภาพในไฟล์หนึ่ง แต่กลับอนุมัตโค้ดเดียวกันในอีกไฟล์ — อะไรคือต้นเหตุ และวิธีแก้ที่เหมาะสมที่สุด?

- A. context window ของโมเดลเล็กเกินกว่าจะเก็บทั้ง 14 ไฟล์ — อัปเกรดไปใช้โมเดลที่มี context window ใหญ่กว่า
- B. แยกการรีวิวเป็น per-file local analysis pass บวกกับ cross-file integration pass แยกอีกหนึ่งรอบ เพื่อเลี่ยง attention dilution
- C. เพิ่ม system prompt ที่แข็งแกร่งขึ้นเน้นย้ำความสำคัญของการรีวิวทุกไฟล์ให้ละเอียดเท่ากัน
- D. ลดจำนวนไฟล์ต่อการรีวิวเหลือ 5 ไฟล์ แล้วประมวลผลเป็น batch เรียงกันครั้งละ 5 ไฟล์

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

สถาปัตยกรรม multi-pass แก้ attention dilution ได้ตรงจุด — per-file pass ทำให้ทุกไฟล์ได้การวิเคราะห์ที่ทุ่มเทและสม่ำเสมอ ส่วน cross-file integration pass จับปัญหา data flow และ pattern ที่ไม่สอดคล้องกัน จึงแก้ทั้งอาการพลาด bug ในไฟล์ท้าย ๆ และการประเมิน pattern ที่ขัดแย้งกัน

ทำไมข้ออื่นผิด

- A — ขนาด context window ไม่ใช่ประเด็น attention dilution เกิดจากการประมวลผลรายการมากเกินไปในรอบเดียว ไม่ว่าโมเดลจะเก็บ context ได้มากแค่ไหน
- C — การปรับ prompt แก้ attention dilution ไม่ได้ ต้นเหตุคือการประมวลผลมากเกินไปในรอบเดียว ซึ่งเป็นปัญหาเชิงโครงสร้างที่ต้องแก้เชิงโครงสร้าง
- D — batching แก้ dilution ภายใน batch ได้ แต่พลาดปัญหาข้าม batch ถ้าไม่มี cross-file integration pass ก็ยังไม่ครอบคลุม data flow และ pattern consistency ทั้ง 14 ไฟล์



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

ทีมต้องเพิ่ม test ให้ legacy codebase ที่มี dependency ซึ่งไม่มี documentation — pattern การทำ task decomposition แบบใดเหมาะสมที่สุด?

A. ~~fixed sequential pipeline~~: วิเคราะห์แต่ละ module, เขียน test, รัน test, รายงานผล

☒ B. dynamic adaptive decomposition: map โครงสร้าง, ค้นหา dependency, จัดลำดับใหม่เมื่อเจอความซับซ้อนที่โผล่ขึ้นมา

C. ~~fixed sequential pipeline~~ บวกกับ context window ใหญ่ขึ้นเพื่อรับมือความซับซ้อน

D. ประมวลผลแต่ละ module อย่างอิสระโดยไม่มี decomposition strategy ใด ๆ

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

การเพิ่ม test ให้ legacy codebase ที่ dependency ไม่มี documentation เป็นงาน investigation แบบ open-ended — scope เติมยังไม่รู้ตั้งแต่ต้น dependency จะโผล่ขึ้นระหว่างสำรวจ dynamic decomposition จึงปรับแผนตามข้อมูลใหม่ที่ค้นพบได้

ทำไมข้ออื่นผิด

- A — fixed pipeline สมมติว่ารู้ step ล่วงหน้า แต่เมื่อ dependency ไม่มี documentation agent กำหนดล่วงหน้าไม่ได้ว่าควร test module ไหนก่อน อาจเจอว่า Module A พึ่ง Module B ทำให้ต้องเปลี่ยนแผน
- C — ขนาด context window ไม่ช่วยเรื่องความยืดหยุ่น ประเด็นคือแผนต้องปรับตามสิ่งที่ค้นพบ ไม่ใช่ว่าโมเดลเก็บข้อมูลไม่พอ
- D — การประมวลผล module อิสระมองข้าม dependency test ของ Module A อาจ fail ถ้า Module B ที่มันพึ่งพายังไม่มี test จึงต้องมี coordination strategy



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

งานใดต่อไปนี้จะเหมาะกับ fixed sequential pipeline (prompt chaining) มากที่สุด?

A. สืบหา root cause ของ production bug ที่เกิดเป็นครั้งคราว (intermittent)

B. สกัด structured data จาก invoice ที่มี format ที่รู้อยู่แล้ว ✓

C. ทำ security audit ของระบบที่ ไม่คุ้นเคย ✗

D. สืบหา feature ของสินค้าคู่แข่งเพื่อทำ market analysis ✗

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

การสกัดข้อมูลจาก invoice มีโครงสร้างที่รู้อยู่แล้ว — field และ format ถูกกำหนดล่วงหน้า step คาดเดาได้: อ่าน invoice, สกัด field, validate format, output structured data fixed pipeline จึงเหมาะกับงานที่มีโครงสร้างและคาดเดาได้

ทำไมข้ออื่นผิด

- A — การสืบ root cause เป็น open-ended สาเหตุยังไม่รู้ ต้องปรับตัวตามเบาะแสที่โผล่ขึ้น dynamic decomposition เหมาะกว่า
- C — security audit ระบบที่ไม่คุ้นเคยต้องอาศัยการสำรวจและปรับตัว ช่องโหว่ที่เจออาจเผยพื้นผิวโจมตีเพิ่มซึ่งเปลี่ยนแผนสอบ
- D — market analysis คือการค้นหาข้อมูลที่ยังไม่รู้ scope ของ feature คู่แข่งไม่ได้รู้ครบล่วงหน้า dynamic decomposition จึงเหมาะกว่า



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

อะไรคือสิ่งที่แยกแยะ attention dilution ออกจากข้อจำกัดด้าน capability ของโมเดล?

A. attention dilution เกิดเฉพาะกับโมเดลเล็ก โมเดลใหญ่ไม่เจอปัญหานี้



B. attention dilution ทำให้ความลึกไม่สม่ำเสมอระหว่างรายการในรอบเดียว ขณะที่ข้อจำกัด capability ให้ผลเยื้องอย่างสม่ำเสมอ

C. attention dilution เกิดจากขนาด context window ที่ไม่พอ



D. attention dilution กระทบเฉพาะงาน code review เท่านั้น ไม่กระทบการวิเคราะห์ประเภทอื่น



⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูกต้อง: Option B

attention dilution มีลักษณะเด่นคือ "ความไม่สม่ำเสมอ" — วิเคราะห์บางรายการละเอียด บางรายการผิวเผิน และ
ประเมิน pattern เดียวกันขัดแย้งกัน ส่วนข้อจำกัด capability จะให้ผลแม่นยำสม่ำเสมอทุกรายการ ความไม่สม่ำเสมอจึงเป็น
สัญญาณบ่งชี้

ทำไมข้ออื่นผิด

- A — attention dilution กระทบโมเดลทุกขนาด เป็นปัญหาเชิงโครงสร้างจากการประมวลผลรายการมากเกินไปในรอบเดียว ไม่ใช่เรื่องพลังของโมเดล
- C — attention dilution เป็นเรื่องการจัดสรร attention ไม่ใช่ขนาด context window การเพิ่ม context window ไม่แก้การกระจาย attention ที่ไม่ทั่วถึง
- D — attention dilution เกิดได้กับทุกงานที่ประมวลผลรายการมากเกินไปในรอบเดียว ทั้ง code review, document analysis, data validation และอื่น ๆ



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

developer แบ่ง 14 ไฟล์เป็นกลุ่มละ 5 ไฟล์เพื่อรีวิว แต่ไม่ได้ cross-file integration pass — วิธีนี้จะพลาดปัญหาแบบใด?

A. bug ภายในไฟล์เดียว — batching ไม่ช่วยเรื่อง local analysis



B. ปัญหา cross-file data flow และ pattern ที่ไม่สอดคล้องกันระหว่างไฟล์ที่อยู่คนละ batch

C. คอขวดด้าน performance ที่เกิดจากการประมวลผลไฟล์เป็น batch



D. ความไม่สอดคล้องด้าน formatting ภายในไฟล์เดียว



⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

batching แก้ attention dilution ภายในแต่ละ batch ได้ แต่ไม่แก้ปัญหาค้าม batch ถ้าไม่มี cross-file integration pass เฉพาะ ปัญหา data flow ระหว่าง module ที่อยู่คนละ batch และการประเมิน pattern ที่ขัดแย้งกันข้าม batch จะไม่ถูกตรวจพบ

ทำไมข้ออื่นผิด

- A — batching กลับช่วยให้ local analysis ดีขึ้นด้วยซ้ำ เพราะให้ attention budget ที่โฟกัสกว่า การวิเคราะห์ per-file หรือ per-batch จับปัญหา local ได้ดีกว่ารอบเดียว
- C — คำถามถามถึงคุณภาพการรีวิว ไม่ใช่ performance และ batching อาจเพิ่ม throughput ด้วยการลดภาระต่อรอบ
- D — ความไม่สอดคล้องด้าน formatting ภายในไฟล์เป็นปัญหา local ที่ batching จัดการได้พอ ปัญหาที่พลาดคือเรื่อง cross-file โดยเฉพาะ